



Universidad de Jaén

Tema 2.5 Jakarta EE, Autorización y Autenticación

Control de estado, proveedores de identidad
y control de acceso

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Esta obra está bajo una [Licencia Creative Commons](#)
[Atribución-CompartirIgual 4.0 Internacional](#).

v 2.3.6

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Objetivos

- Comprender el concepto de **estado en una aplicación web** y saber utilizar diferentes **mecanismos para mantener el estado** de una sesión
- Conocer los aspectos básicos del modelo de **autorización y autenticación** de la plataforma JEE
- Saber **restringir el acceso a recursos** de una aplicación web mediante reglas de control de acceso
- Conocer diferentes medios de **gestionar credenciales de usuario** en un servidor de aplicaciones JEE
- Conocer diferentes **métodos de autenticación** en aplicaciones web y saber utilizar métodos basados en usuario y clave
- Conocer y saber utilizar los fundamentos del **control de acceso de forma programada** en una aplicación web JEE



Control de estado en una aplicación web

Gestión del estado en aplicaciones web

- El protocolo **HTTP no mantiene el estado** entre peticiones consecutivas
- **Mecanismos para conservar el estado** de una aplicación entre sucesivas llamadas con un mismo cliente:
 - **Cookies:**
 - Fragmentos de información enviados al cliente (encabezado) que almacena localmente
 - El cliente los devuelve al servidor con cada nueva petición HTTP al mismo dominio
 - El cliente los elimina si alcanzan su caducidad
 - **Almacenamiento en cliente** (*LocalStorage*, *SessionStorage*, ...)
 - Almacenamiento clave/valor en navegador vinculado a cada sitio visitado
 - Accesible desde Javascript
 - **Sesión:**
 - Mecanismo para almacenar el estado de la conexión de un usuario con el servidor
 - *Almacenamiento en servidor:* suele utilizar una cookie con un token para identificar al cliente, e.g. cookie de sesión. Habitual en aplicaciones web con alta interacción con el cliente
 - *Almacenamiento en cliente:* *LocalStorage*/encabezados específicos firmados/cifrados con información del estado que será utilizada en el servidor, e.g. JWT. Uso en aplicaciones con accesos esporádicos o sin soporte de cookies

Cookies

● Gestión:

- Creados/manipulados por el servidor, aunque también desde el cliente
- El cliente los envía automáticamente como encabezados al servidor cuando hace peticiones HTTP al mismo host/dominio y ruta asociada a la cookie
- El cliente puede desactivarlas, editarlas e incluso borrarlas...
- La legislación española obliga a autorizarlos: Artículo 4 RD 13/2012

● Composición:

Name	Value	Domain	Path	Expires / Max-Age	Size	HTTP	Secure	SameSite
JSESSIONID	s-IPVtyCio5T2BXFO2M3u...	sica2.cica.es	/	1969-12-31T23:59:...	58	✓	✓	

- **Identificador (Name)**
- **Contenido (Value, ≈ 4KB entre todas las cookies de un mismo sitio)**
- **Host o Dominio web** del que procede y/o asigna. e.g. www.ujaen.es o .ujaen.es
- **Ruta(Path)** a la que se enviará la cookie (debe haberse obtenido desde ellas). Por defecto la url de la AppWeb
- **Caducidad** en segundos antes de eliminarse en cliente. 0 para eliminar, -1 para duración hasta cierre del navegador
- **Http-only**, accesible o no desde JS. Evitar que pueda enviarse a otros servidores (e.g. ataques XSS)
- **Secure**, el navegador solo la enviará si es sobre una conexión segura TLS (e.g. ataques MiTM)
- **SameSite**, el navegador solo la enviará al servidor si la página que hace el envío se ha descargado del dominio indicado. Evita operaciones no autorizadas desde sitios maliciosos (e.g. ataques de tipo CSRF)

5

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Sesiones gestionadas en servidor

- **Definición:** Una sesión engloba todas las peticiones realizadas desde un cliente a la aplicación web mientras no se cierre el navegador o se finalice ésta expresamente

● Motivación:

- Los cookies o *LocalStorage* se pueden "modificar" en el cliente.
- Durante la sesión es necesario **almacenar información específica en el servidor**
 - Identificador del usuario y roles de acceso
 - Valores introducidos desde formularios: productos seleccionados (carrito), secuencias de formularios, etc.
 - Datos temporales sobre la conexión: tiempo conectado, última página visitada, etc.

6

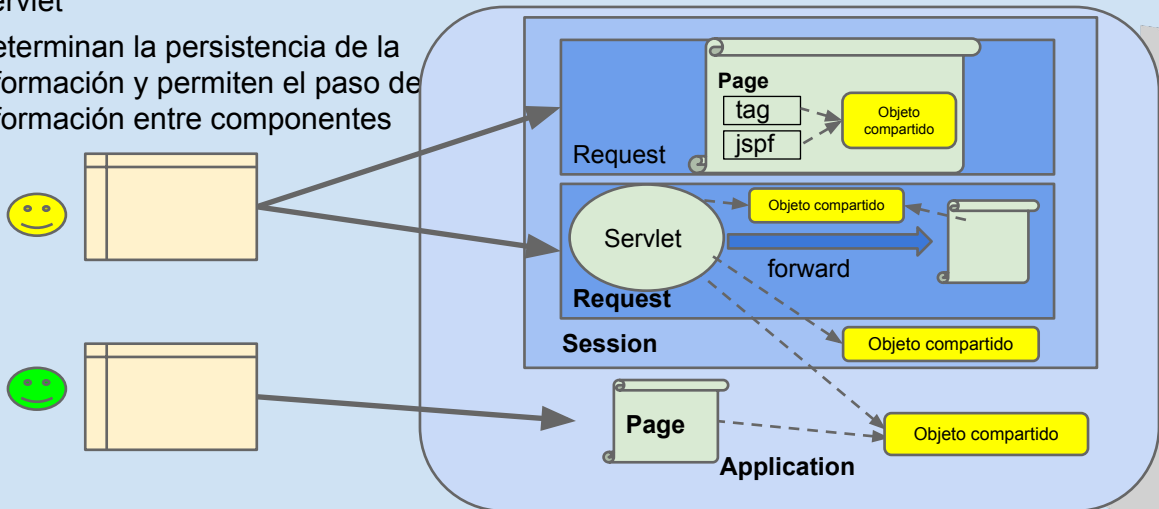
Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Contextos en una aplicación web

Definidos en la especificación
Servlet

Determinan la persistencia de la
información y permiten el paso de
información entre componentes



7

Gestión de sesiones en el servidor

- El **servidor almacena los datos** de cada sesión en un lugar (**contexto web**) específico. Puede configurarse, e.g. ficheros, BBDD,...
- El contexto de sesión **se borra cuando finaliza** la sesión. e.g. se cierra el navegador
- ¿Cómo se asigna el contexto de sesión a una petición?
 - **Cookie específica**, **JSESSIONID**, identificador único asociada a una sesión de usuario.
 - El navegador puede deshabilitar el envío de cookies!
 - **Parámetro específico**: enviado por el cliente en todas las peticiones como query-string o campo oculto en formulario
 - `<input type="hidden" name="JSESSIONID" value="xxx">`
 - sólo en envíos de formularios
 - `http://host/webapp/servlet?JSESSIONID=xxx&...`
(NO recomendado por cuestiones de seguridad. PUEDE DESACTIVARSE)
 - Hay que añadirlo a todas las URLs que se generen
 - En **Servlets**: `response.encodeUrl("dir_web")`
 - En **JSP**: usando jstl, `<c:url value="dir_web"/>`

8

Configuración de la cookie de sesión en JEE

- Puede configurarse de forma declarativa en /WEB-INF/web.xml

```
<session-config>
  <session-timeout>
    30                                <!-- Time in minutes to expire after the last request -->
  </session-timeout>
  <cookie-config>
    <!-- Prevents client side scripting from accessing/manipulating session cookie. -->
    <http-only>true</http-only>
    <secure>true</secure> <!-- true to force send over TLS connection -->
  </cookie-config>
  <tracking-mode>COOKIE</tracking-mode> <!-- This disables URL rewriting. -->
</session-config>
```

- Por ahora no es posible establecer el atributo *SameSite* de forma directa en web.xml.
- Puede hacerse modificando la cookie enviada al cliente mediante un *Filter*

Name	Value	Domain	Path	Expires / Max-Age	Size	HTTP	Secure	SameSite
JSESSIONID	s-IPVtyCio5T2BXFO2M3u...	sica2.cica.es	/	1969-12-31T23:59:...	58	✓	✓	

9

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Gestión de sesiones en Servlets/JSP

- El contenedor gestiona la sesión automáticamente a partir del id de sesión obtenido: cookie o id en url
- **Obtener el contexto de sesión**
 - **Servlet**, `HttpSession session=request.getSession();`
 - **JSP/EL**, `${sessionScope.atributo}` en scriptlets
- **Gestión de información en contexto de sesión**
 - **Servlets**
 - `Object session.getAttribute("clave")`
 - OJO, debe hacerse un casting al tipo adecuado
 - `session.setAttribute("clave",valor)`, almacenar/reemplazar un dato(objeto) asociado a una clave
 - `session.removeAttribute("clave")`, eliminar un valor
 - `session.invalidate()`, eliminar información de sesión y descartar JSESSIONID. e.g. al realizar un *logout* de la aplicación
 - **JSP/EL**
 - Solo lectura `${sessionScope.clave}` o `${sessionScope["clave"]}`

10

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Ejemplo de atributos de sesión

```
private void processRequest (...) {  
    HttpSession session = request.getSession();  
    Integer visits=(Integer)session.getAttribute("visits");  
    if (visits==null)  
        visits=0; //No previous visits  
    session.setAttribute("visits", ++visits);  
    //Contexts does not preserve type (casting needed)  
    String sess_user=(String)session.getAttribute("user");  
    String param_user=request.getParameter("user");  
  
    if (param_user!=null && !param_user.trim().equals("")) {  
        sess_user=param_user.trim();  
        session.setAttribute("user", sess_user); //new value  
    }  
    if (sess_user==null) //session default value  
        session.setAttribute("user", "Desconocido");  
    request.setAttribute("conn_date",  
        new Date(session.getCreationTime() ));  
    request.getRequestDispatcher("/WEB-INF/nombre.jsp")  
        .forward(request, response);  
}
```

Hola Desconocido

elige un nuevo nombre

Nuevo nombre:
Conectado desde: Tue Mar 26 17:05:50 CET 2013
Accesos: 1

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Session Sample</title>
```

```
</head>
```

```
<body>
```

```
<h1>Hola ${sessionScope.user}</h1>
```

```
<h2>elige un nuevo nombre</h2>
```

```
<form method="POST">
```

```
    Nuevo nombre:
```

```
    <input name="user" value="${user}">
```

```
    <input type="submit" name="enviar"  
        value="Cambiar">
```

```
</form>
```

```
<footer>Conectado desde: ${conn_date}
```

```
</footer>
```

```
<br>Accesos: ${sessionScope.visits}
```

```
</body>
```

```
</html>
```

Hola Maria

elige un nuevo nombre

Nuevo nombre:
Conectado desde: Tue Mar 26 17:05:50 CET 2013
Accesos: 2

11

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Gestión de sesión en JSF

• Acceso a contexto de sesión

- Utilizar beans con ámbito de persistencia **@SessionScoped**
- Desde vistas: acceso con EL desde vistas **#\${sessionScope.variable}**
- En controlador, se puede obtener a partir del objeto `HttpServletRequest`. Disponible con **@Inject**

```
@Inject HttpServletRequest request;
```

```
...
```

```
request.getSession().invalidate(); //e.g. at logout
```

• Ejemplo

```
@Named ( value="prefs")
```

```
@SessionScoped
```

```
public class Preferencias implements Serializable {
```

```
    private String color = "white";
```

```
    public Preferencias () {};
```

```
    public String getColor () {
```

```
        return color;
```

```
    }
```

```
<!-- perfil.xhtml -->
```

```
<h1>Preferencias de usuario</h1>
```

```
<div> Color de fondo: #{prefs.color}</div>
```

12

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Autorización y Autenticación en Jakarta EE

Conceptos básicos

- **Conceptos**
 - **Autenticación:** identificar y validar que un usuario es quien dice ser
 - **Autorización:** permitir el acceso de un usuario a un recurso según sus privilegios
- Son **tareas habituales en aplicaciones web:**
 - **Procesos complejos** que complican la implementación de la aplicación
 - **Propensas** a errores
 - Críticas para la **seguridad** de una aplicación
 - Diferentes **tecnologías:** Formularios, OAuth2, 2FA, *Remember me*, ...
 - **Pueden variar** de forma independiente a la aplicación
- **Solución**
 - Desvincular la autenticación y autorización de la lógica de control
 - La plataforma o *framework* deben proporcionar utilidades
 - JEE define diferentes APIs para facilitar estas tareas
 - Nos centraremos en la capa de presentación web

Mecanismos de autenticación y autorización en JEE

- **JAAS**, *Java Authentication and Authorization Service*, mecanismo estándar de la plataforma Java. Basado en módulos genéricos o personalizados.
- **JASPIC/JACC**, *Java Authentication Service Provider Interface for Containers/Java Authorization Contract for Containers*, permite crear **proveedores de autenticación o autorización** personalizados para aplicaciones web. Complejos de implementar y son parte del servidor
- **Realms (dominios)**, proveedores de seguridad proporcionados por los servidores de aplicaciones sobre almacenes de credenciales habituales: ficheros, BBDD, LDAP, etc. Poca capacidad de personalización, desvinculados de aplicación web, gestión dependiente del servidor [+info](#)
- **Jakarta Security 3.0**, Simplifica el proceso de creación de proveedores de seguridad desde las propias aplicaciones (**utilizaremos este enfoque**)
 - JEE 8+, Soteria como implementación de referencia (RI)
 - Define **Interfaces para autenticación** y almacenes de identidad (**Identity Stores**)
 - Dispone de algunas **implementaciones**, e.g. *IdentityStores* como *database*, *ldap*, ...

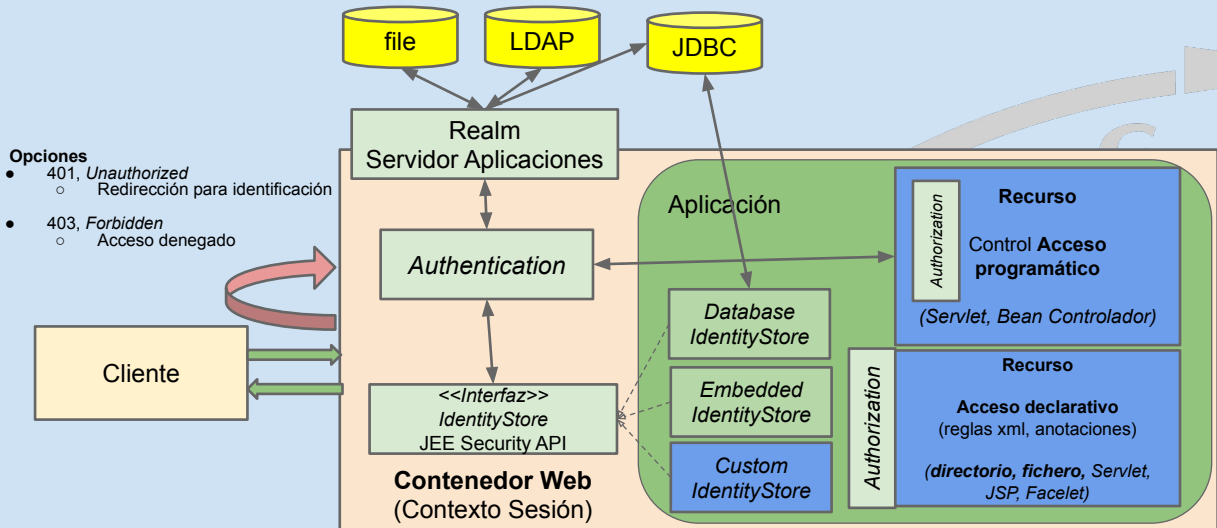
15

Control de acceso a aplicaciones web JEE

- **Tipos de control de acceso en la capa Web JEE**
 - **Declarativo:**
 - **Reglas específicas** con restricciones de acceso a recursos en `web.xml`
 - **ventajas:** no acoplado a la implementación, control acceso a recursos estáticos (carpetas o ficheros)
 - **Anotaciones** en Servlets o Servicios Web (REST)
 - **inconveniente:** poca granularidad en ciertas situaciones
 - **En implementación:**
 - Control mediante código en los propios recursos
 - **ventaja:** mayor granularidad
 - **inconveniente:** acoplamiento y dificultad de implementación
 - Ambos enfoques pueden **complementarse**

16

Modelo de seguridad en aplicaciones web JEE



17

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Proceso de autenticación

- **Fases:**
 - El cliente trata de acceder a un recurso con una restricción de acceso
 - Si **usuario está autenticado** y
 - Tiene acceso (rol autorizado), el contenedor procesa la petición
 - No tiene acceso (rol no autorizado) → acceso denegado (403 forbidden)
 - Si el **usuario no está autenticado**, se solicita al cliente que lo identifique (auth. básica) o lo redirige a un formulario de acceso (auth. formulario)
 - Si la **autenticación es errónea**:
 - tipo básico: tras varios intentos se muestra un mensaje de error (401 Unauthorized)
 - tipo formulario: se redirige al cliente a una página de error personalizada.

18

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Almacenes de identidad en JEE Security API

- Beans que implementan la **interfaz IdentityStore**
 - Validan credenciales** de usuario, e.g. user/pass, token, ...
 - Obtienen los **roles de acceso** de usuarios
 - Suelen acceder a un repositorio/servicio para acceso a datos de usuarios
- Implementaciones predeterminadas:**
 - Basadas en anotaciones
 - `@DatabaseIdentityStoreDefinition`,
 - `@LdapIdentityStoreDefinition`
 - `@EmbeddedIdentityStoreDefinition` (en RI Soteria, necesaria en pom.xml)

• Roles

- Determinan el acceso a los recursos (RBAC, *Role Based Access Control*)
- Los usuarios se **asocian a roles** para tener **acceso a los recursos** correspondientes. También pueden clasificarse en **grupos**
- Los roles válidos deben declararse.**

Opción 1. En web.xml

```
<security-role>
  <role-name>USUARIOS</role-name>
</security-role>
<security-role>
  <role-name>ADMINISTRADORES</role-name>
</security-role>
```

Opción 2.- Anotación en algún objeto con anotaciones CDI/Servlet

```
@DeclareRoles({"ADMINISTRADORES","USUARIOS"})
@ApplicationScoped
@FacesConfig //Enable JSF
public class AppConfig {
```

Almacén de identidad en memoria

- Útil para pruebas de autenticación (**uso inseguro en producción**)
- Es necesaria la implementación de referencia (RI) de JEE Security API: Soteria (disponible en Payara/Glassfish):

```
<dependency>
  <groupId>org.glassfish.soteria</groupId>
  <artifactId>jakarta.security.enterprise</artifactId>
  <version>3.0.3</version>
  <scoped>provided</scoped>
</dependency>
```

- Ejemplo de definición de credenciales

```
@EmbeddedIdentityStoreDefinition({
  @Credentials( callerName = "admin", password = "secret1", groups = {"ADMINISTRADORES"}),
  @Credentials( callerName = "user", password = "secret2", groups = {"USUARIOS"})
})
@ApplicationScoped // Needed for CDI
@FacesConfig //Enable Jakarta Faces
public class AppConfig {
}
```

Control acceso (autorización) a recursos declarativo

- Usado principalmente para recursos estáticos, Facelets, JSP
- Configuración en web.xml
- Reglas de restricción de acceso

Ejemplo: Sólo los usuarios que pertenecen al rol de administradores pueden acceder a la carpeta /admin/ usando conexiones seguras SSL/TLS

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Acceso admin</web-resource-name>
    <url-pattern>/admin/*</url-pattern>
    <!-- otras rutas -->
    <http-method>GET</http-method><!-- otros métodos -->
  </web-resource-collection>
  <!-- otras colecciones de recursos/métodos -->
  <auth-constraint>
    <role-name>ADMINISTRADORES</role-name>
    <role-name><!-- otros roles --></role-name>
  </auth-constraint>
  <user-data-constraint> <!-- Opcional: requiere acceso seguro SSL/TLS -->
    <transport-guarantee>CONFIDENTIAL</transport-guarantee>
  </user-data-constraint>
</security-constraint> <!-- más reglas -->
```

21

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Control de acceso declarativo con anotaciones

- Solo se pueden utilizar en código
- **@ServletSecurity**
 - Solo para Servlets (JEE6+)
 - Opciones:
 - **@HttpMethodConstraint**: restricciones por método HTTP (get, post, ...)
 - **@HttpConstraint**: restricciones para resto de peticiones (por defecto)
- **@RolesAllowed**, **@PermitAll**
 - Servicios Jakarta REST → Se estudiará en tema 4.3
 - Jakarta Enterprise Beans → Se estudiará en Desarrollo de Aplicaciones Empresariales

```
@WebServlet("/clientes/*") //Opción básica
@ServletSecurity(@HttpConstraint( rolesAllowed = "Admins" ) )
public class ClienteController extends HttpServlet { ... }

@WebServlet("/clientes/*") //Opción detallada
@ServletSecurity(
  value=@HttpConstraint( rolesAllowed = {"Employes", "Admins"} ),
  httpMethodConstraint={
    @HttpMethodConstraint(value="POST",rolesAllowed={"Admins"},
      // only allowed if SSL/TSL connection
      transportGuarantee = TransportGuarantee.CONFIDENTIAL),
    @HttpMethodConstraint(value="PUT",
      // if not roles defined then DENY any user
      emptyRoleSemantic=ServletSecurity.EmptyRoleSemantic.DENY),
  })
public class ClienteController extends HttpServlet { ... }
```

22

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Mecanismos de autenticación en aplicaciones web con JEE Security

- Definen **cómo el servidor recibirá las credenciales** que deben validarse
- Por cuestiones de **seguridad** todos los envíos de credenciales deben hacerse por canales HTTP seguros TLS
- **Algunos tipos:**
 - **Básico:** el navegador muestra un cuadro de diálogo para introducir el usuario y la clave. Empleado principalmente para API Rests
 - **Formulario estándar,** el *servidor de aplicaciones* recoge las credenciales desde un formulario web de la aplicación
 - **Formulario con autenticación personalizada,** la *aplicación* recoge las credenciales, autentica al usuario y lo registra en el servidor
 - **Autenticación externa con OpenID, JEE10+.** delega en un servidor externo el proceso de autenticación, e.g. Google, Github, Twitter...

23

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Configuración de mecanismos de Autenticación JEE Security API (I)

Método: mediante anotaciones en algún bean accesible

- **Autenticación básica**

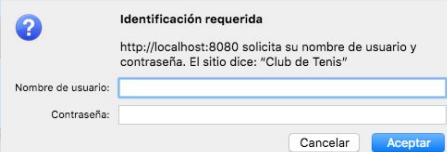
```
@EmbeddedIdentityStoreDefinition({
    @Credentials( callerName = "admin", password = "secret1",
        groups = {"ADMINISTRADORES"}),
})
@BasicAuthenticationMechanismDefinition(
    realmName = "Club de Tenis"
)
@ApplicationScoped // Needed for CDI
@FacesConfig
public class AppConfig { }
```

Empleado en aplicaciones web muy básicas y API REST

- **Autenticación mediante formulario estándar**

```
@FormAuthenticationMechanismDefinition(
    loginToContinue = @LoginToContinue(
        loginPage = "/login.jsf",
        errorPage = "/login.jsf?error",
        useForwardToLogin = false
    )
)
```

- **error** parámetro que usamos para mostrar un mensaje de error
- anulamos *forward* para PRG después de autenticar al usuario



24

Desarrollo de Aplicaciones Web. jrbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Formulario estándar de autenticación

- Puede ser de cualquier tipo de vista
 - html, facelet, jsp ...
- Debe utilizar unos **parámetros y acción específicos** para ser detectado por el sistema de autenticación
- **Ejemplo:** facelet, aunque con formulario html (no usa etiquetas jsf)

```
<h2>Identificación de usuario </h2>
<h:panelGroup rendered="#{param.error!=null}">
  <p class="alert alert-danger">Error de autenticación</p>
</h:panelGroup>
<form action="_j_security_check" method="POST">
  Usuario: <input type="text" name="_j_username"/>
  Clave: <input type="password" name="_j_password"/>
  <input type="submit" name="Identificarse"/>
</form>
```

Importante: el acceso al formulario lo realiza el servidor cuando necesita identificar a un usuario. Para ello, hacer que el botón de login de la aplicación apunte a una url que esté protegida forzando así la redirección a la vista del formulario

- **Inconvenientes**, no permite utilizar funcionalidades JSF, e.g. mensajes de error personalizados, ajax, ...

25

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Configuración de mecanismos de Autenticación JEE Security API (II)

- Tipo Formulario personalizado

```
@EmbeddedIdentityStoreDefinition({
  @Credentials(callerName = "admin", password = "secret1",
    groups = {"ADMINISTRADORES"}),
})
@CustomFormAuthenticationMechanismDefinition(
  loginToContinue = @LoginToContinue(
    loginPage = "/customlogin.jsf",
    errorPage = "",
    useForwardToLogin = false
  )
)
@ApplicationScoped // Needed for CDI
@FacesConfig
public class AppConfig { }
```

- reutilizamos el formulario para mostrar errores desde jsf
- anulamos *forward* para PRG después de autenticar al usuario

26

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Formulario personalizado de autenticación

- **Requisitos**

- El controlador debe recoger las credenciales pasarlas al servidor de aplicaciones para autenticarlo

- **Ejemplo:**

```
<h2>Identificación de usuario</h2>
<h:messages class="alert alert-danger"/>
<h:form >
    Usuario:<h:inputText value="#{clubLoginController.username}" />
    Clave: <h:inputSecret value="#{clubLoginController.password}" />
    <h:commandButton value="Login" action="#{ clubLoginController.login }"/>
</h:form>
```

- **Ventajas**

- Podemos hacer uso de formularios JSF, bean validation, ajax...

27

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Control acceso en Facelets/Controladores

- **SecurityContext** permite acceder a información sobre el proceso de autenticación. Disponible con **@inject** en controladores

- **authenticate(request, response, AuthenticationParameters)**, reenvía la petición actual al servidor con credenciales suministradas
- **getCallerPrincipal()**, objeto que representa al usuario. Usar método **getName()** para conocer su identificador. **`#{facesContext.externalContext.userPrincipal.name}`**
 - También puede obtenerse en controlador mediante **@Inject Principal usuario;**
- **isCallerInRole("rolename")** comprueba si el usuario actual pertenece o no al rol/es indicados, **`#{facesContext.externalContext.isUserInRole("ADMINISTRADORES")}`**
- **hasAccessToWebResource("/url", http-method)**, comprueba si el usuario actual tiene acceso a url local con un método http, e.g. GET

- **HttpServletRequest** dispone de métodos para comprobar y controlar el usuario identificado. Disponible con **@inject** en JSF

- **getRemoteUser()**, Nombre del usuario identificado. **`null`** si no lo está. **`#{request.remoteUser}`**
- **getUserPrincipal()**, objeto que representa al usuario. Usar método **getName()** para conocer su identificador. **`#{request.userPrincipal.name}`**
- **isUserInRole("rol")**, true si el usuario pertenece al rol indicado. **`#{pageContext.request.isUserInRole('rol')}`**, **`#{request.isUserInRole('rol')}`**
- **logout()**, finaliza la conexión del usuario (suele acompañarse de **`session.invalidate()`**)

28

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Controlador para autenticación programática

```
@ViewScoped @Named
public class ClubLoginController implements Serializable{
```

```
    @Inject FacesContext fc;
    @Inject SecurityContext sc; //JEE security API access
    @Inject ExternalContext ec; //JEE web server
    @Inject HttpServletRequest req; //Current http request
    //view-model
    @Size(min=4, max=16,
        message = "id entre {min} y {max} caracteres")
    private String username;

    @NotEmpty (message = "Introduzca una clave válida")
    private String password;
```

```
    public String getUsername() {
        return username;
    }
    public void setUsername(String login) {
        this.username = login;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```

```
    public String login() { //Login form action
        String view="";

        //Prepare data for programmatic authentication
        AuthenticationParameters authData = new AuthenticationParameters();
        Credential cred = new UsernamePasswordCredential( username, password);
        authData.credential(cred).newAuthentication(true);

        HttpServletResponse resp =(HttpServletResponse) ec.getResponse();

        //Programmatic authentication: pass auth request to server
        if ( sc.authenticate(req, resp, authData)==AuthenticationStatus.SUCCESS) {
            if (req.isUserInRole("ADMINISTRADORES")) {
                view="/admin/index?faces-redirect=true";
            } else {
                view="/cliente/index?faces-redirect=true";
            }
            logger.log(Level.INFO,"Usuario autenticado");
        } else {
            fc.addMessage("", new FacesMessage(
                FacesMessage.SEVERITY_WARN, "Error de autenticación", ""));
            logger.log(Level.WARNING,"Error de autenticación");
        }
        return view;
    }
}
```

29

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Gestión de Logout

Acciones:

plantilla.xhtml Botón opcional de logout en Menú

Menú

Logout admin

Inicio

```
<h:panelGroup rendered="#{(not empty request.userPrincipal)}">
    <li class="nav-item">
        <h:form styleClass="">
            <h:commandLink styleClass="nav-link active bg-warning"
                action="#{clubLoginController.logout}"
                value="Logout #{request.userPrincipal.name}"/>
        </h:form>
    </li>
</h:panelGroup>
```

Si tenemos un *Principal* personalizado
podemos acceder a sus propiedades, e.g.
`#{request.userPrincipal.usuario.nombre}`

```
@ViewScoped
@Named
public class ClubLoginController implements Serializable{
    @Inject FacesContext fc;
    @Inject SecurityContext sc; //JEE security API access
    @Inject ExternalContext ec; //JEE web server
    @Inject HttpServletRequest req; //Current http request
```

// ...

```
    public String logout() throws ServletException {
        req.logout(); //Clear Principal info
        req.getSession().invalidate();
        //ec.invalidateSession();

        return "/index?faces-redirect=true";
    }
}
```

30

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Almacén de identidad en base de datos en JEE Security

- **Requisitos**

- Datasource definido mediante JNDI (e.g. en web.xml)
- Requiere un método de cifrado/verificación de claves

- **Ejemplo**

```
@DatabaseIdentityStoreDefinition(  
    dataSourceLookup = "java:global/jdbc/gestClub", // Datasource connection  
    callerQuery = "select clave from Usuarios where usuario = ?", // retrieve hashed password  
    groupsQuery = "select rol from Roles where usuario = ?", // retrieve user roles  
    hashAlgorithm = Pbkdf2PasswordHash.class, // Encryption/Validation algorithm on EE security  
    hashAlgorithmParameters = {  
        "Pbkdf2PasswordHash.Algorithm=PBKDF2WithHmacSHA512",  
        "Pbkdf2PasswordHash.Iterations=3072",  
        "Pbkdf2PasswordHash.SaltSizeBytes=64"  
    }  
)  
...  
@ApplicationScoped // Needed for CDI  
@FacesConfig  
public class AppConfig {  
    ...  
}
```

Ejemplo de organización de credenciales y roles

Usuarios		Roles	
<u>usuario</u>	VARCHAR(50)	<u>usuario</u>	VARCHAR(50)
clave	VARCHAR(250)	<u>rol</u>	VARCHAR(50)
...		...	

31

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Almacén de Identidad personalizado

- **Requisitos**

- Cualquier bean que implementa la interfaz `IdentityStore`
- Al menos debe implementar el método `validate`
- Debe consultar credenciales válidas y devolver roles de usuarios
- Devuelve la información del usuario (al menos el identificador) accesible en objeto de clase `Principal` (o derivada)

- **Ejemplo**

```
@ApplicationScoped  
public class ClubIdentityStore implements IdentityStore {  
  
    @Inject private ClubAuthService authService;  
    @Inject @DAOMap private ClienteDAO clientesDAO;  
  
    public CredentialValidationResult validate(UsernamePasswordCredential upc) {  
  
        String username = upc.getCaller();  
        String password = upc.getPasswordAsString();  
  
        Cliente cliente = clientesDAO.buscarByNIF(username); //Look up using user login  
  
        if (cliente != null &&  
            authService.verifyPassword(password, cliente.getClaveCifrada()) ) {  
            //Get roles for valid user...  
            Set<String> roles = new HashSet<>(asList("USUARIOS"));  
            //Authenticate user in Application Server and pass custom User information  
            return new CredentialValidationResult( new ClubPrincipal(cliente), roles);  
            //return new CredentialValidationResult( username, roles);  
        }  
  
        return INVALID_RESULT; //Credentials invalid  
    }  
}
```

```
class ClubPrincipal  
    extends CallerPrincipal {  
  
    private final Cliente usuario;  
  
    public ClubPrincipal(Cliente usu) {  
        super( usu.getDni() );  
        this.usuario = usu;  
    }  
    public Cliente getUsuario() {  
        return usuario;  
    }  
}
```

Acceso a información personalizada en vistas con:
`#{request.userPrincipal.usuario.nombre}`

32

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Cifrado y verificación de claves con EE Security

- Las claves de usuarios deben tener un **cifrado robusto**:
 - Algoritmo hash seguro
 - Longitud adecuada
 - Utilizar sal diferente para cada clave
 - El proceso de generación debe ser "costoso"
- JEE Security proporciona:
 - Interfaz **PasswordHash** para algoritmos de cifrado
 - Implementación **Pbkdf2PasswordHash**. DI con **@Inject**
 - Necesario en **DatabasIdentityStore**

```
@ApplicationScoped  
public class ClubAuthService {
```

```
    @Inject private Pbkdf2PasswordHash cipher;
```

```
    @PostConstruct  
    public void init() {  
        //Configure encryption algorithm  
        Map<String, String> config = new HashMap<>();  
        config.put("Pbkdf2PasswordHash.Iterations", "3072");  
        config.put("Pbkdf2PasswordHash.Algorithm",  
            "PBKDF2WithHmacSHA512");  
        config.put("Pbkdf2PasswordHash.SaltSizeBytes", "64");  
        cipher.initialize( config );  
    }
```

```
    @Override  
    public String encryptPassword(String pass) {
```

```
        String encryptedPass = cipher.generate(pass.toCharArray());  
        return encryptedPass;  
    }
```

```
    @Override  
    public boolean verifyPassword( String password, String encryptedPass ) {  
        return cipher.verify(password.toCharArray(), encryptedPass);  
    }
```

33

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Conclusiones

- Las **sesiones** permiten **mantener el estado** de la interacción de un usuario en una aplicación web
- La **cookie de sesión** identifica unívocamente a la sesión de un usuario en una aplicación web
- En general, **la autorización y el control de acceso en aplicaciones web debe estar desvinculado** de la implementación de la propia aplicación.
- La gestión de **credenciales de acceso** en aplicaciones web JEE debe delegarse en alguno de los mecanismos estándar de la plataforma JEE: Módulos JASPIC, Realms en servidor o módulos basados en *Security API*.
- Los dominios, **Realms**, son mecanismos de gestión de credenciales de acceso proporcionados por un servidor. Su gestión varía según el servidor.
- La capa web en JEE dispone de **mecanismos declarativos y a nivel de programación** para la **gestión centralizada del control de acceso** a los recursos de la aplicación. Estos mecanismos **deben complementarse** para definir la política de control de acceso de la aplicación

34

Desarrollo de Aplicaciones Web. irbalsas@ujaen.es. <http://tiny.cc/dawuja>. 2025



Bibliografía

- JakartaEE Web Profile Security, en *Jakarta EE Tutorial*. Eclipse Foundation, 2024. Disponible en <https://jakarta.ee/learn>
- Arjan Tijms, Bauke Scholtz. Chapter 10 Security in *The definitive guide to Jakarta Faces in Jakarta EE 10: building Java-Based web applications*. Second Edition. Apress, 2022. Disponible en [[Recurso electrónico](#)]
- D. R. Heffelfinger, Securing JEE applications, en *Java EE 8 Application Development*. Packt Publishing, 2017. [[Recurso electrónico](#)]
- *JakartaEE Security Specification*. Version 3.0. Eclipse Foundation, 2022. Disponible en <https://jakarta.ee/specifications/security/3.0/>